

Reporte de evidencia

Para la presente practica se llevó a cabo un micro invernadero inteligente, capaz de monitorear y controlar de manera automática algunas de las variables ambientales que influyen directamente en el desarrollo de las plantas.

El sistema utiliza un ESP32 como unidad de control, junto con sensores destinados a medir la temperatura y las condiciones de iluminación del entorno. A partir de estas mediciones, el sistema controla diferentes actuadores, principalmente un ventilador para regular la temperatura y un LED de potencia para complementar la iluminación.

La propuesta se desarrolla dentro del contexto de la programación orientada a objetos por lo que se busca no solamente realizar la lectura de sensores, sino establecer una relación entre las variables medidas, clases y métodos de los diversos objetos de estudio y acciones de control ejecutadas por los actuadores.

Objetivos

- Medir la temperatura del interior del invernadero inteligente usando un LM35
- Medir las condiciones del interior con un sensor LDR
- Implementar un sistema de ventilación usando PWM
- Establecer condiciones automáticas de operación para los actuadores.
- Implementar comunicación mediante el puerto serial.
- Generar información de telemetría para conocer el estado del sistema.
- Evaluar el comportamiento del sistema mediante condiciones de operación.
-

Descripción general del sistema

El sistema está compuesto por cuatro elementos principales:

1. Unidad de procesamiento: ESP32.
2. Sensores: LM35 para temperatura y LDR para iluminación.
3. Actuadores: ventilador y LED de potencia.
4. Interfaz de comunicación: puerto serial.

El ESP32 realiza periódicamente la lectura de los sensores, luego procesa los valores obtenidos y determina la acción correspondiente sobre el ventilador y el sistema de iluminación.

La información obtenida también puede enviarse mediante comunicación serial, permitiendo observar el estado actual del sistema y, posteriormente, desarrollar una interfaz HMI o conectar el micro invernadero con una computadora u otro sistema de supervisión.

Sensores:

Sensor de temperatura LM35

El LM35 es utilizado para obtener la temperatura del micro invernadero. Su salida es una señal analógica proporcional a la temperatura.

En el programa, la lectura se realiza mediante el ADC del ESP32:

```
int valorADC = analogRead(PIN_TEMPERATURA);  
float voltaje = (valorADC * 3.3) / 4095.0;  
float temp = voltaje * 100.0;
```

Considerando una resolución de ADC de 12 bits, la lectura puede tomar valores entre 0 y 4095.

El cálculo utilizado supone una referencia aproximada de 3.3 V y la característica del LM35 de aproximadamente 10 mV/°C.

Sensor LDR

El LDR permite obtener una estimación de la intensidad luminosa presente en el sistema.

La lectura también se realiza mediante una entrada analógica del ESP32:

```
valorLDR = analogRead(PIN_LDR);
```

El valor obtenido se encuentra dentro del rango correspondiente al ADC utilizado.

A diferencia del sensor de temperatura, el LDR no proporciona directamente una magnitud física expresada en lux en la implementación actual. Por lo tanto, en esta primera versión se utiliza principalmente como variable de referencia para determinar el nivel de iluminación artificial requerido.

Actuadores

Ventilador

El ventilador constituye el actuador encargado de reducir la temperatura cuando ésta supera el límite establecido por el algoritmo.

El sistema utiliza PWM para controlar el actuador:

```
#define FRECUENCIA_PWM 5000
```

Con una resolución de 8 bits, el ciclo de trabajo puede variar entre:

0 → 255

donde:

- 0 corresponde al 0 %.
- 255 corresponde aproximadamente al 100 %.

En la versión provisional, el control implementado es de tipo ON/OFF:

```
if (temperatura > 30.0) {  
    dutyVentilador = 255;  
} else {  
    dutyVentilador = 0;  
}
```

Por lo tanto, cuando la temperatura supera los 30 °C, el ventilador se activa al 100 %. Cuando la temperatura se encuentra en 30 °C o menos, el ventilador se desactiva.

Iluminación artificial

El LED de potencia funciona como fuente de iluminación artificial.

A diferencia del ventilador, el LED utiliza actualmente un control proporcional basado en la lectura del LDR:

```
dutyLED = map(
  valorLDR,
  0,
  4095,
  255,
  0
);
```

Esto genera una relación inversa:

Menor iluminación detectada → mayor iluminación artificial

Mayor iluminación detectada → menor iluminación artificial

Posteriormente, esta estrategia puede modificarse para establecer un nivel objetivo de iluminación y desarrollar un control más preciso.

Asignación de entradas y salidas

Elemento	Dispositivo	Pin ESP32	Tipo
Sensor de temperatura	LM35	GPIO 34	ADC
Sensor de iluminación	LDR	GPIO 32	ADC
Ventilador	Motor DC	GPIO 18	PWM
Iluminación	LED de potencia	GPIO 19	PWM

Los canales PWM utilizados son:

Actuador	Canal PWM	Frecuencia	Resolución
Ventilador	0	5000 Hz	8 bits
LED	1	5000 Hz	8 bits

Estrategia de control

La estrategia provisional está basada en dos reglas principales.

Control de temperatura

La variable controlada es:

Temperatura [$^{\circ}\text{C}$]

El actuador correspondiente es:

Ventilador [% PWM]

La lógica actual puede expresarse como:

$$u_{\text{ventilador}} = \begin{cases} 100\% & T > 30^{\circ}\text{C} \\ 0\% & T \leq 30^{\circ}\text{C} \end{cases}$$

Este comportamiento corresponde a un control de dos posiciones (ON/OFF).

Control de iluminación

La variable medida es el valor del LDR:

LDR [0–4095]

y el actuador es:

LED [0–255 PWM]

La estrategia implementada establece una relación inversa entre ambas variables. En términos conceptuales:

$$LDR \downarrow \Rightarrow PWM_{LED} \uparrow$$

$$LDR \uparrow \Rightarrow PWM_{LED} \downarrow$$

Esto permite compensar parcialmente una reducción de iluminación ambiental mediante el aumento de la iluminación artificial.

Telemetría

Una característica importante del sistema es la generación de información de telemetría.

El programa proporciona información sobre:

- Temperatura.
- Valor del LDR.
- Duty cycle del ventilador.
- Duty cycle del LED.

Por ejemplo, el sistema puede generar información estructurada en formato JSON:

```
{  
  "temperatura": 28.50,  
  "ldr": 2140,  
  "ventilador": 0,  
  "led": 122  
}
```

Este formato resulta útil para una futura integración con:

- Una aplicación de escritorio.
- Una interfaz HMI.
- Una página web.
- Una base de datos.
- Sistemas de IoT.
- Plataformas de monitoreo.

Interfaz de operación

La comunicación serial permite consultar información y enviar instrucciones al microcontrolador.

Actualmente se contemplan comandos como:

- TEMP
- LDR
- STATUS
- JSON
- FAN ON
- FAN OFF
- LED OFF

Esta funcionalidad representa la primera aproximación a una HMI de consola, similar al concepto empleado en el simulador de reactor proporcionado como referencia.

En versiones posteriores se puede ampliar la interfaz para incorporar diferentes modos de operación, por ejemplo:

- Modo Manual
- Modo Automático
- Modo Pruebas

Esto permitiría separar claramente las funciones de operación normal, intervención del usuario y validación del sistema.

Seguridad e interlocks

Uno de los conceptos principales tomado del simulador de reactor es el uso de un interlock de seguridad de máxima prioridad.

En el caso del reactor de referencia, el interlock se activa cuando:

$$T > 85^{\circ}C$$

$$P > 12 \text{ Bar}$$

y fuerza los actuadores a una condición segura.

En el micro invernadero, este concepto deberá adaptarse a las variables reales del sistema. Por ejemplo, puede establecerse un límite máximo de temperatura que obligue al ventilador a funcionar independientemente del modo operación seleccionado.

Código implementado en el ESP32

A continuación, se presenta el código completo utilizado en la versión final del invernadero inteligente:

```
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11

#define LDR_PIN 34
#define LED_PIN 2

#define ENA 26
#define IN1 27
#define IN2 14

DHT dht(DHTPIN, DHTTYPE);

bool sistemaEncendido = false;

int temperaturaObjetivo = 30;

int ldrOscuro = 0;
int ldrLuz = 3109;

const int pwmFreq = 5000;
const int pwmResolution = 8;

void setup() {

    Serial.begin(115200);

    dht.begin();

    pinMode(LDR_PIN, INPUT);

    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);

    ledcAttach(LED_PIN, pwmFreq, pwmResolution);
    ledcAttach(ENA, pwmFreq, pwmResolution);
}
```

```

digitalWrite(IN1, LOW);
digitalWrite(IN2, LOW);

ledcWrite(LED_PIN, 0);
ledcWrite(ENA, 0);

Serial.println();
Serial.println("=====");
Serial.println("      SISTEMA DE CONTROL ESP32");
Serial.println("=====");
Serial.println("Comandos:");
Serial.println("encender");
Serial.println("apagar");
Serial.println("leer");
Serial.println("ajustar 30");
Serial.println();
Serial.println("Temperatura permitida: 0 a 50 C");
Serial.println("=====");
}

void loop() {

    revisarComandos();

    if (sistemaEncendido) {

        controlarTemperatura();
        controlarIluminacion();

    } else {

        detenerVentilador();
        ledcWrite(LED_PIN, 0);
    }

    delay(200);
}

void revisarComandos() {

    if (Serial.available() > 0) {

        String comando = Serial.readStringUntil('\n');

```

```
comando.trim();
comando.toLowerCase();

if (comando == "encender") {

    sistemaEncendido = true;

    Serial.println();
    Serial.println("Sistema ENCENDIDO");

}

else if (comando == "apagar") {

    sistemaEncendido = false;

    detenerVentilador();
    ledcWrite(LED_PIN, 0);

    Serial.println();
    Serial.println("Sistema APAGADO");

}

else if (comando == "leer") {

    mostrarDatos();

}

else if (comando.startsWith("ajustar")) {

    int espacio = comando.indexOf(' ');

    if (espacio != -1) {

        int nuevaTemperatura =
            comando.substring(espacio + 1).toInt();

        if (nuevaTemperatura >= 0 &&
            nuevaTemperatura <= 50) {

            temperaturaObjetivo = nuevaTemperatura;

        }

    }

}
```

```

        Serial.print("Temperatura objetivo: ");
        Serial.print(temperaturaObjetivo);
        Serial.println(" C");

    } else {

        Serial.println(
            "Error: usa un valor entre 0 y 50 C"
        );
    }

} else {

    Serial.println("Ejemplo: ajustar 30");
}
}

else {

    Serial.println("Comando no reconocido");
}
}
}

void controlarTemperatura() {

    float temperatura = dht.readTemperature();

    if (isnan(temperatura)) {

        Serial.println("Error al leer DHT11");

        detenerVentilador();

        return;
    }

    int pwmVentilador = 0;

    if (temperatura <= temperaturaObjetivo) {

        pwmVentilador = 0;

    }
}

```

```

else if (temperatura >= 50) {

    pwmVentilador = 255;

}

else {

    pwmVentilador = map(
        temperatura * 10,
        temperaturaObjetivo * 10,
        500,
        120,
        255
    );

    pwmVentilador = constrain(
        pwmVentilador,
        120,
        255
    );
}

if (pwmVentilador > 0) {

    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);

    ledcWrite(
        ENA,
        pwmVentilador
    );

} else {

    detenerVentilador();
}
}

void controlarIluminacion() {

    int valorLDR = analogRead(LDR_PIN);

    int pwmLED = map(

```

```

    valorLDR,
    ldrOscuro,
    ldrLuz,
    255,
    0
);

pwmLED = constrain(
    pwmLED,
    0,
    255
);

ledcWrite(
    LED_PIN,
    pwmLED
);
}

void detenerVentilador() {

    ledcWrite(ENA, 0);

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
}

void mostrarDatos() {

    float temperatura = dht.readTemperature();
    float humedad = dht.readHumidity();

    int valorLDR = analogRead(LDR_PIN);

    if (isnan(temperatura) ||
        isnan(humedad)) {

        Serial.println("Error al leer DHT11");

        return;
    }

    int pwmLED = 0;
    int pwmVentilador = 0;

```

```
if (sistemaEncendido) {

    pwmLED = map(
        valorLDR,
        ldrOscuro,
        ldrLuz,
        255,
        0
    );

    pwmLED = constrain(
        pwmLED,
        0,
        255
    );

    if (temperatura > temperaturaObjetivo) {

        if (temperatura >= 50) {

            pwmVentilador = 255;

        } else {

            pwmVentilador = map(
                temperatura * 10,
                temperaturaObjetivo * 10,
                500,
                120,
                255
            );

            pwmVentilador = constrain(
                pwmVentilador,
                120,
                255
            );
        }
    }
}

int porcentajeLED =
    map(
        pwmLED,
```

```
    0,  
    255,  
    0,  
    100  
);  
  
int porcentajeVentilador =  
    map(  
        pwmVentilador,  
        0,  
        255,  
        0,  
        100  
    );  
  
Serial.println();  
Serial.println("===== DATOS =====");  
  
Serial.print("Sistema: ");  
  
if (sistemaEncendido) {  
    Serial.println("ENCENDIDO");  
}  
else {  
    Serial.println("APAGADO");  
}  
  
Serial.print("Temperatura: ");  
Serial.print(temperatura);  
Serial.println(" C");  
  
Serial.print("Humedad: ");  
Serial.print(humedad);  
Serial.println(" %");  
  
Serial.print("Temperatura objetivo: ");  
Serial.print(temperaturaObjetivo);  
Serial.println(" C");  
  
Serial.print("LDR: ");  
Serial.println(valorLDR);
```

```

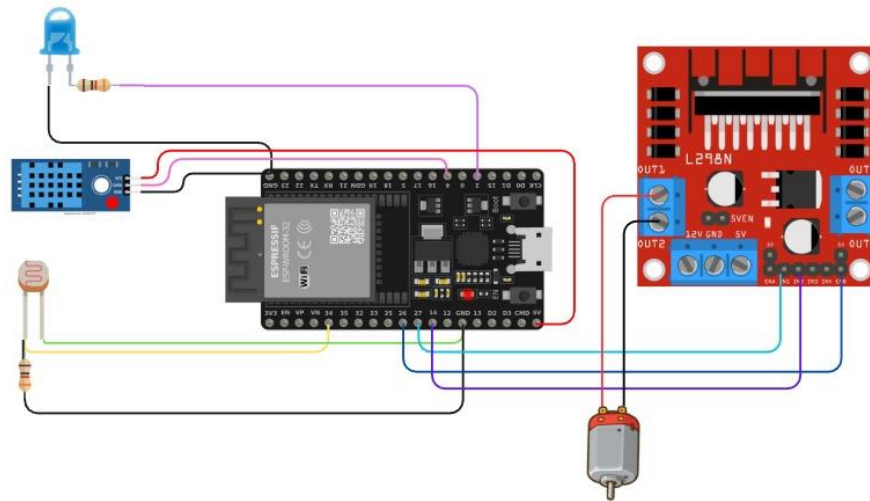
Serial.print("LED: ");
Serial.print(porcentajeLED);
Serial.println(" %");

Serial.print("Ventilador: ");
Serial.print(porcentajeVentilador);
Serial.println(" %");

Serial.println("=====");
}

```

Diagrama eléctrico de conexiones:



Descripción del circuito:

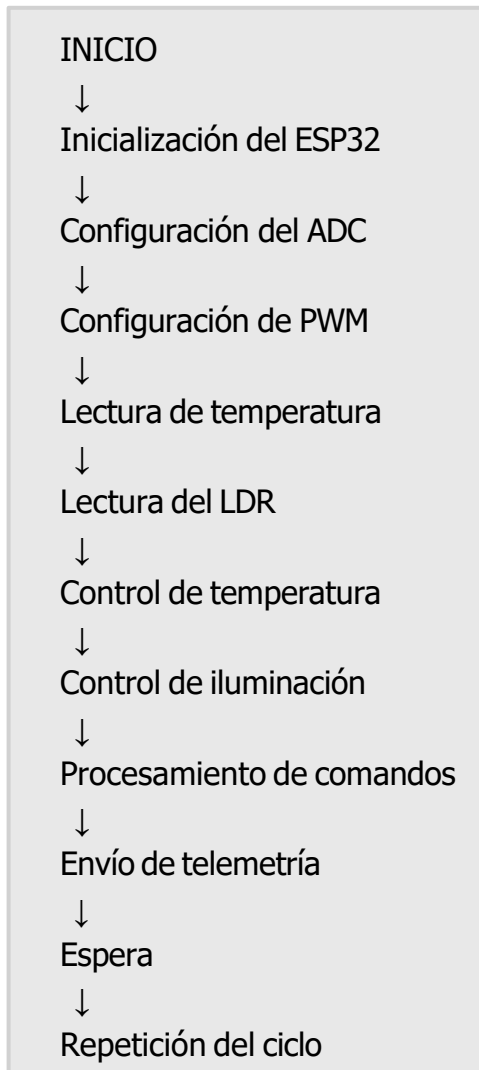
El circuito corresponde al sistema de control del micro invernadero inteligente. El ESP32 se encarga de recibir la información de los sensores conectados al sistema. El sensor DHT11 permite obtener los valores de temperatura y humedad, mientras que el LDR detecta las condiciones de iluminación.

El ESP32 controla los actuadores del sistema. El LED se utiliza como fuente de iluminación y su intensidad puede variar con el PWM, el motor que funciona como ventilador se conecta con el módulo L298N, el cual maneja su activación, el circuito permite supervisar las condiciones ambientales y mejores condiciones dentro del invernadero.

La arquitectura propuesta sería:

Lectura de sensores → Evaluación de seguridad → Control normal → Actuadores

De esta forma, las condiciones críticas tendrían prioridad sobre cualquier comando



Funcionamiento del programa

El ciclo principal del programa sigue aproximadamente la siguiente secuencia:

El ciclo se ejecuta aproximadamente cada 1 segundo, debido al uso de:

```
delay(1000);
```

Estado actual del prototipo

La implementación presentada corresponde a una versión provisional 2.0.0 del sistema.

Actualmente se cuenta con:

- Lectura de temperatura.
- Lectura de iluminación.
- Control automático del ventilador.
- Control automático del LED.
- PWM para los actuadores.
- Comunicación serial.
- Telemetría en texto.
- Telemetría en formato JSON.
- Comandos manuales básicos.

Sin embargo, todavía existen elementos que pueden incorporarse en las siguientes versiones:

- Modo manual completo.
- Modo automático configurable.
- Modo de pruebas.
- Interlock de seguridad.
- Límites configurables.
- Histéresis para evitar conmutaciones frecuentes del ventilador.
- Filtrado de las lecturas de sensores.
- Registro histórico de datos.
- Interfaz gráfica.
- Comunicación inalámbrica.
- Control más avanzado de temperatura e iluminación.

Videos:

- Reporte físico:

<https://drive.google.com/file/d/1ZaUwifEZFI3cNQkXyP4Esv-azHy4AexZ/view?usp=sharing>

- Reporte simulación:

https://drive.google.com/file/d/1DNg08fiyaJWURBNwcuTosx1kq_O5E1rU/view?usp=sharing